sql-logic-mapping

SQL Logic Mapping Prompt Chain

Use Case

Use this chain when a human analyst needs Edge Copilot to turn messy business logic, reporting notes, or stakeholder language into a precise SQL-ready implementation plan.

The chain is for mapping and validation before SQL. It should not produce final SQL until the mapping is complete and unresolved assumptions have been answered.

Required Local Preparation

- Gather the business request, acceptance criteria, sample output, and any known filters or date windows.
- Prepare table names, field names, relationship notes, and compressed schema context when available.
- Identify the expected output grain, such as one row per customer, account, order, day, month, product, or business event.
- Keep examples of edge cases, exclusions, and known contradictory rules ready.

Prompt Sequence

1. `01-business-logic-intake.md`: collect business rules and block premature SQL generation.
2. `02-field-and-table-mapping.md`: produce the reusable logic-map table.
3. `03-grain-and-join-validation.md`: validate grain, joins, duplicates, and many-to-many risks.
4. `04-sql-implementation-plan.md`: convert the completed map into a SQL-ready implementation plan.
5. `05-review-for-ambiguity.md`: review the plan for unresolved assumptions and clarification questions.

Related Procedure Closet

- None yet. This chain is a prompt-chain-only workflow.

Related Reference Drawers

- `references/oracle-sql/sections/oracle_idioms.md`

SQL Logic Mapping Chain - 01 Business Logic Intake

Act as a SQL business analyst translating unclear stakeholder requirements into a precise SQL implementation plan.

This is a mapping workflow, not a query-writing workflow. Do not write SQL yet. First, help me clarify the business logic and identify missing context.

Business request:

```
<paste messy business logic, reporting request, acceptance criteria, examples, or stakeholde
```

Known context:

```
<paste known tables, fields, date windows, filters, output columns, sample rows, or schema n
```

Return:

1. Restated business objective.
2. Candidate input requirements.
3. Candidate output columns.
4. Missing table relationships.
5. Missing or unclear date windows.
6. Ambiguous field names or contradictory filters.
7. Clarification questions that must be answered before SQL can be written.

Block SQL generation if the mapping inputs are incomplete.

SQL Logic Mapping Chain - 02 Field And Table Mapping

Continue the SQL logic mapping workflow.

Using the business request and context already provided, create a structured logic-map table before writing any SQL.

Use exactly these columns:

input requirement	business rule	table	field	join key	filter condition	derived calculation	aggregation level	output column	ambiguity or assumption

For each row, map one business requirement or output element to the likely SQL implementation components.

Explicitly flag:

1. Missing table relationships.
2. Missing or unclear fields.
3. Ambiguous field names.
4. Missing date windows.
5. Contradictory filters.
6. Rules that need derived calculations.
7. Requirements that have no known source table.

Do not write SQL. If any row has unresolved ambiguity, ask clarification questions after the table.

SQL Logic Mapping Chain - 03 Grain And Join Validation

Continue the SQL logic mapping workflow.

Validate the current logic-map table for output grain and join safety before any SQL implementation plan is written.

Current logic map:

```
<paste the current logic-map table here>
```

Schema or relationship notes:

```
<paste PK/FK notes, table relationship notes, row-count notes, or compressed schema context
```

Return:

1. Expected output grain.
2. Grain of each source table, if inferable.
3. Join path between mapped tables.
4. Missing table relationships.
5. Duplicate-producing join risks.
6. Many-to-many join risks.
7. Aggregation or deduplication needed before joins.
8. Clarification questions required before SQL can be written.

Do not write SQL if grain, join path, or many-to-many handling remains unclear.

SQL Logic Mapping Chain - 04 SQL Implementation Plan

Continue the SQL logic mapping workflow.

Create a SQL-ready implementation plan from the completed logic map. Do not write final SQL unless every mapping, grain, join, date window, and filter assumption is resolved.

Completed logic map:

```
<paste completed logic-map table here>
```

Validated grain and join notes:

```
<paste grain, join-path, deduplication, aggregation, and clarification answers here>
```

Return:

1. SQL generation readiness: ready or blocked.
2. Required base tables and aliases.
3. Join sequence and join keys.
4. Pre-aggregation or deduplication steps.
5. Filter plan, including date windows.
6. Derived calculation plan.
7. Aggregation and grouping level.
8. Output column plan.
9. Remaining blockers, if any.

If blocked, ask only the clarification questions needed to unblock the plan.

SQL Logic Mapping Chain - 05 Review For Ambiguity

Continue the SQL logic mapping workflow.

Review the SQL-ready implementation plan for unresolved ambiguity before any final query is written.

Implementation plan:

```
<paste SQL-ready implementation plan here>
```

New stakeholder answers or corrections:

```
<paste any new clarifications, changed rules, or reviewer comments here>
```

Return:

1. Ambiguities resolved by the new answers.
2. Ambiguities still unresolved.
3. Contradictory filters or business rules that remain.

4. Any missing date windows, field names, or table relationships.
5. Any duplicate-producing or many-to-many join risks still present.
6. Whether final SQL generation is ready or blocked.
7. Final clarification questions if blocked.

Do not generate final SQL unless you explicitly mark the plan as ready.